

Funciones de Python

Clase: Python para ciencias de la atmósfera

Lucía Pedraza Díaz

Repaso

```
ciudades = ['Qro', 'CDMX', 'Ensenada']  
  
for ciudad in ciudades:  
    print(ciudad)
```

Variables

Objetos

ciudades → ['Qro', 'CDMX', 'Ensenada']
.....

1 ciudad

Repaso

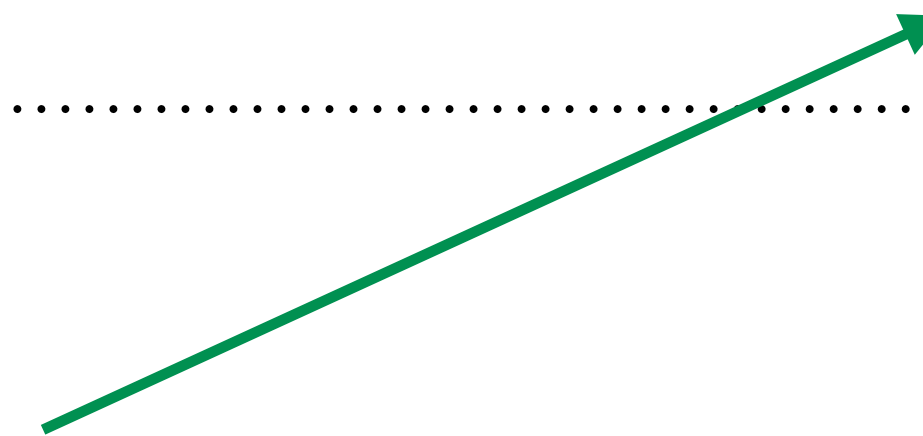
```
ciudades = ['Qro', 'CDMX', 'Ensenada']  
  
for ciudad in ciudades:  
    print(ciudad)
```

Variables

Objetos

ciudades → ['Qro', 'CDMX', 'Ensenada']
.....

2 ciudad



Repaso

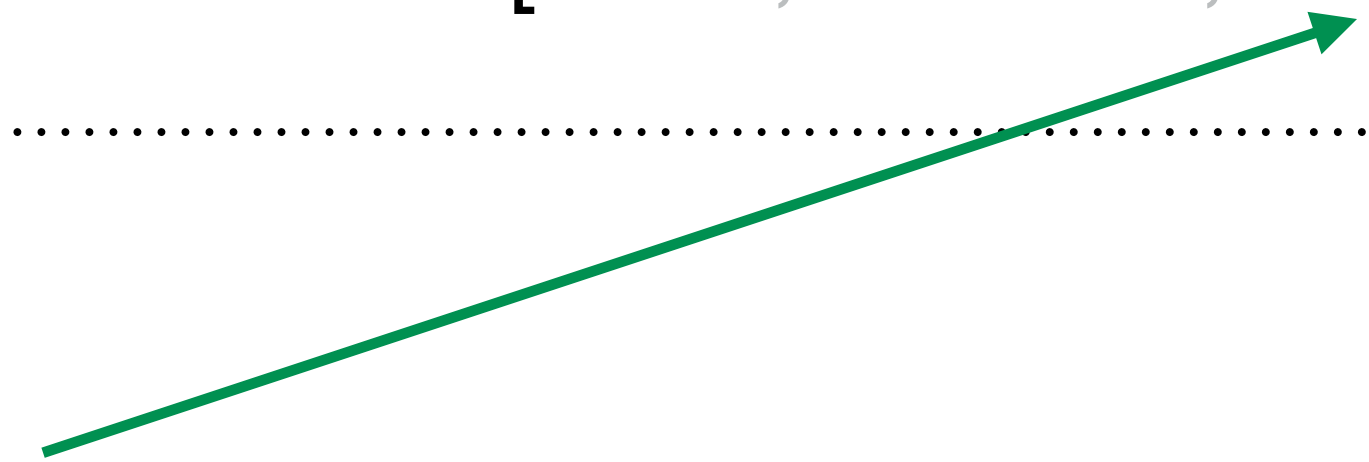
```
ciudades = ['Qro', 'CDMX', 'Ensenada']  
  
for ciudad in ciudades:  
    print(ciudad)
```

Variables

Objetos

ciudades → ['Qro', 'CDMX', 'Ensenada']
.....

3 ciudad



```
l_tuplas = [(1,2,3), (4,5,6), (7,8,9)]  
  
for n1, n2, n3 in l_tuplas:  
    print(n1)  
    print(n2)  
    print(n3)
```

Variables

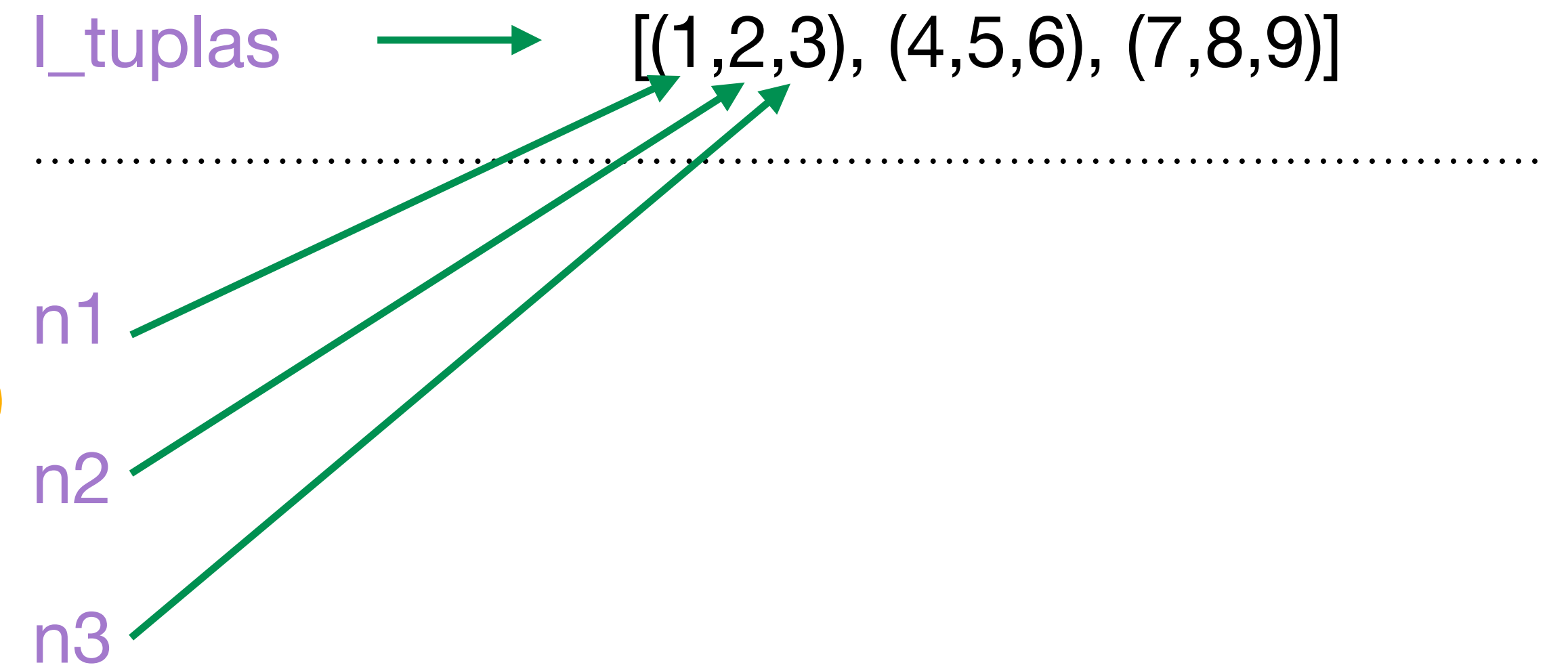
Objetos

l_tuplas → [(1,2,3), (4,5,6), (7,8,9)]
.....

```
l_tuplas = [(1,2,3), (4,5,6), (7,8,9)]  
  
for n1, n2, n3 in l_tuplas:  
    print(n1)  
    print(n2)  
    print(n3)
```

Variables

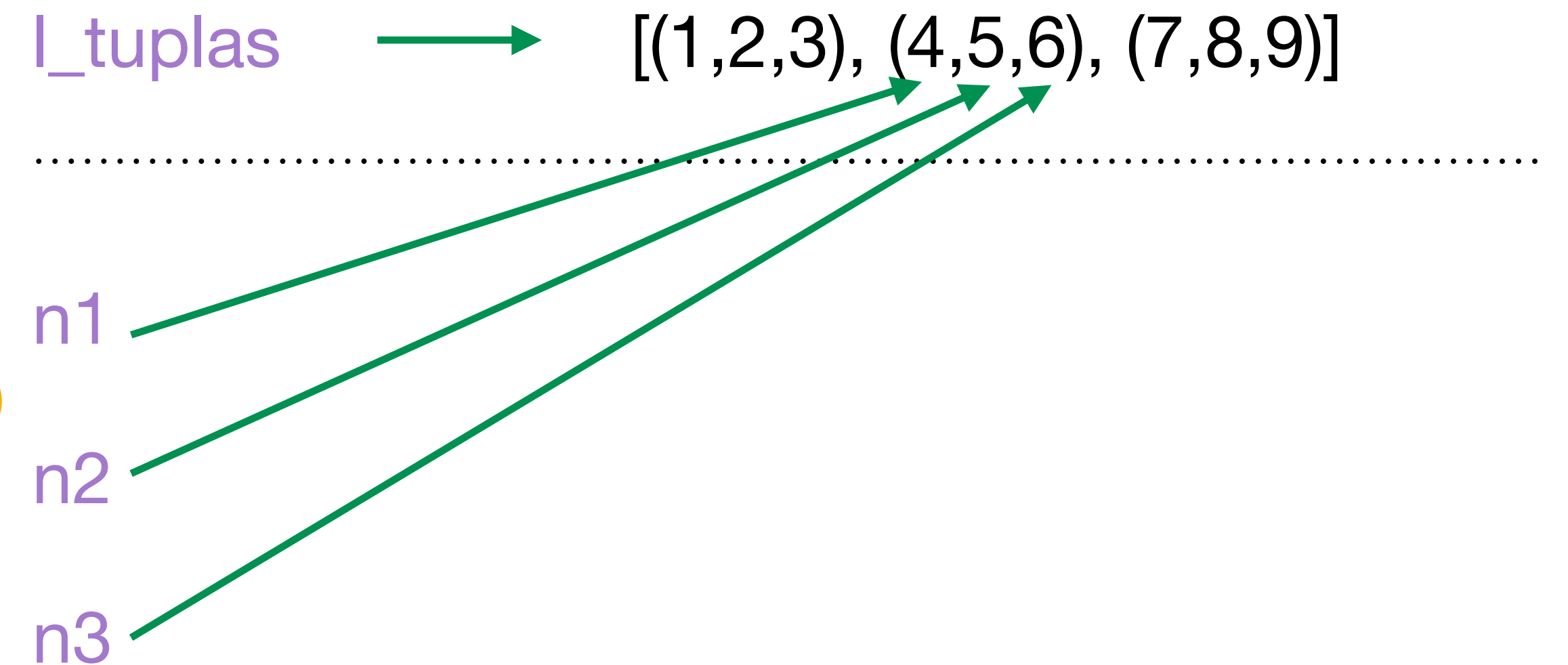
Objetos



```
l_tuplas = [(1,2,3), (4,5,6), (7,8,9)]  
  
for n1, n2, n3 in l_tuplas:  
    print(n1)  
    print(n2)  
    print(n3)
```

Variables

Objetos



2

¿Cómo creamos funciones?

palabra clave que
encabeza la definición de
una función

Nombre de la función

Siempre se ponen
paréntesis. Pueden incluir
o no parámetros.

```
4 def saludo():  
5     | pass  
6  
7     saludo()
```

Bloque de código que se
ejecuta al llamar a la
función. Debe estar
indentado.

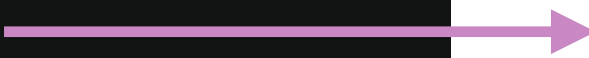
¿Cómo creamos funciones?

```
4  def saludo():  
5      |    pass  
6  
7  saludo()
```

La función se define
(se genera) pero no se
ejecuta

¿Cómo creamos funciones?

```
4  def saludo():  
5  |      pass  
6  
7  saludo()
```

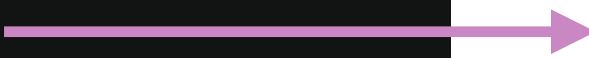


Llamada a la función. SIEMPRE hay que llamarla incluyendo los paréntesis. Los paréntesis pueden o no contener argumentos.

La función se ejecuta cuando la llamamos.

¿Cómo creamos funciones?

```
4  def saludo():  
5  |      pass  
6  
7  saludo()
```



Llamada a la función. SIEMPRE hay que llamarla incluyendo los paréntesis. Los paréntesis pueden o no contener argumentos.

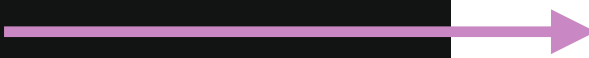
Ejemplos:

- `len(lista)`, `suma(lista)`, `lista.append(algo)`

La función se ejecuta cuando la llamamos.

¿Cómo creamos funciones?

```
4  def saludo():
5  |      pass
6
7  saludo()
```



Llamada a la función. SIEMPRE hay que llamarla incluyendo los paréntesis. Los paréntesis pueden o no contener argumentos.

Ejemplos:

- `len(lista)`, `suma(lista)`, `lista.append(algo)`

Funciones integradas de python
Puedes pasar cualquier tipo de objeto
como argumento

`append()` es un método de las listas. Los
métodos también son funciones pero
solo se pueden usar con un objeto
específico.

La función se ejecuta cuando la llamamos.

Valor de retorno

Las funciones
SIEMPRE tienen un
valor de retorno.

¿Cuál es el valor de
retorno de ésta
función?



```
4  def saludo():  
5  |      pass  
6  
7  saludo()
```

Valor de retorno

Las funciones SIEMPRE tienen un valor de retorno.

¿Cuál es el valor de retorno de ésta función?



```
4  def saludo():  
5      |    pass  
6  
7  saludo()
```

Si NO especificamos el valor de retorno, la función regresa **None**.

- * None es un objeto único que existe una sola vez en todo el programa. Representa la ausencia de valor.
- * Siempre hay que compararlo con el operador de identidad `is`

Valor de retorno

¿Cómo confirmamos
que el valor de
retorno es None?

Tenemos que guardar
el valor de retorno en
una variable.



```
def saludo():  
    pass  
  
valor_retorno = saludo()  
print(valor_retorno)
```

Ejercicios

```
def saludo():  
    print('Hola mundo')
```

1. Llama a la función
y corre el código.
¿Cuál es la salida del
código?

Ejercicios

```
def saludo():  
    print('Hola mundo')
```

1. Llama a la función y corre el código. ¿Cuál es la salida del código?
2. ¿Cuál es el valor de retorno de la función?

Valor de retorno

Podemos indicar el valor de retorno con return.



```
def saludo():  
    return('Hola mundo')
```

Ejercicios

```
def saludo():  
    print('Hola mundo')
```

1. Llama a la función y corre el código. ¿Cuál es la salida del código?
2. ¿Cuál es el valor de retorno de la función?

```
def saludo():  
    return('Hola mundo')
```

1. Llama a la función y corre el código. ¿Cuál es la salida del código?
2. ¿Cuál es el valor de retorno de la función?

Llamada a la función sin paréntesis

```
def saludo():  
    return('Hola mundo')  
  
valor_retorno = saludo()
```

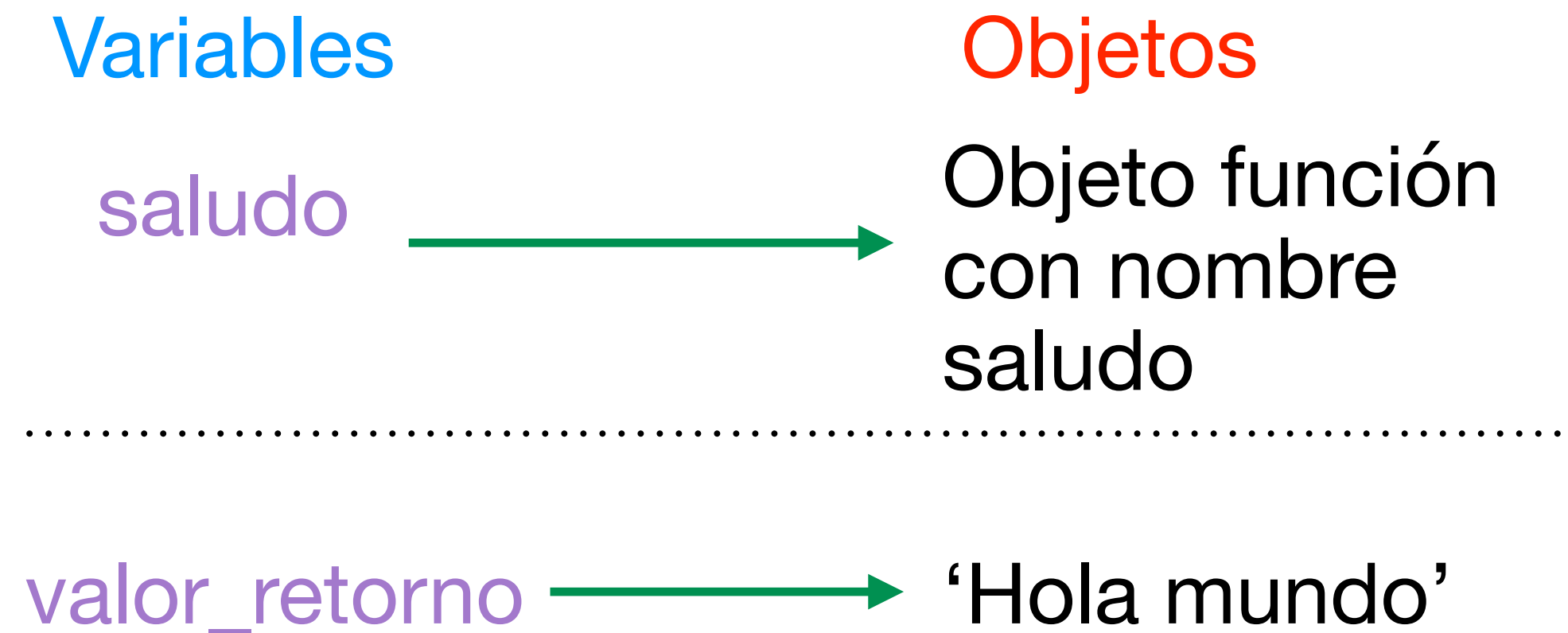
Llamada a la
función

```
def saludo():  
    return('Hola mundo')  
  
valor_retorno = saludo
```

¿Qué pasa si
se nos olvidan
los paréntesis?

Llamada a la función sin paréntesis

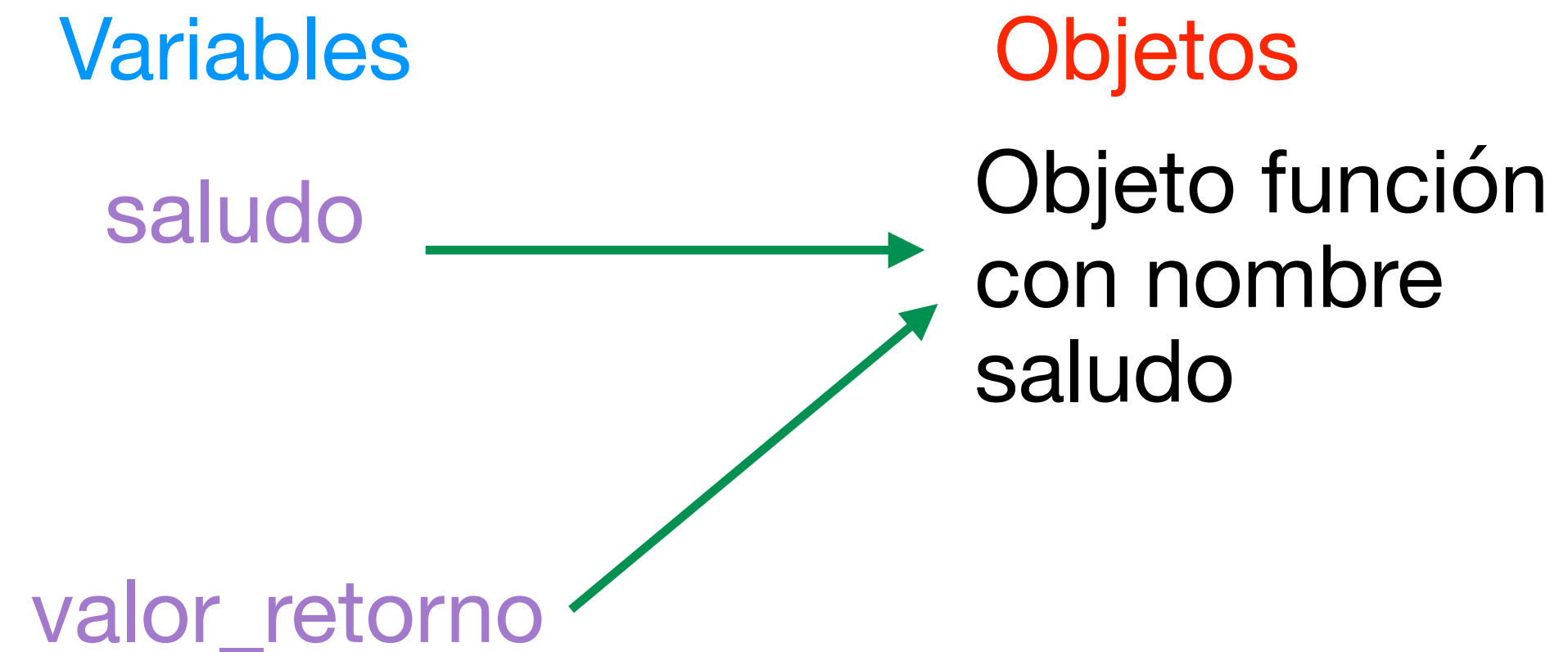
```
def saludo():  
    return('Hola mundo')  
  
valor_retorno = saludo  
  
print(valor_retorno)
```



¿Qué pasa si
se nos olvidan
los paréntesis?

Llamada a la función sin paréntesis

```
def saludo():  
    return('Hola mundo')  
  
valor_retorno = saludo  
  
print(valor_retorno)
```



¿Qué pasa si se nos olvidan los paréntesis?

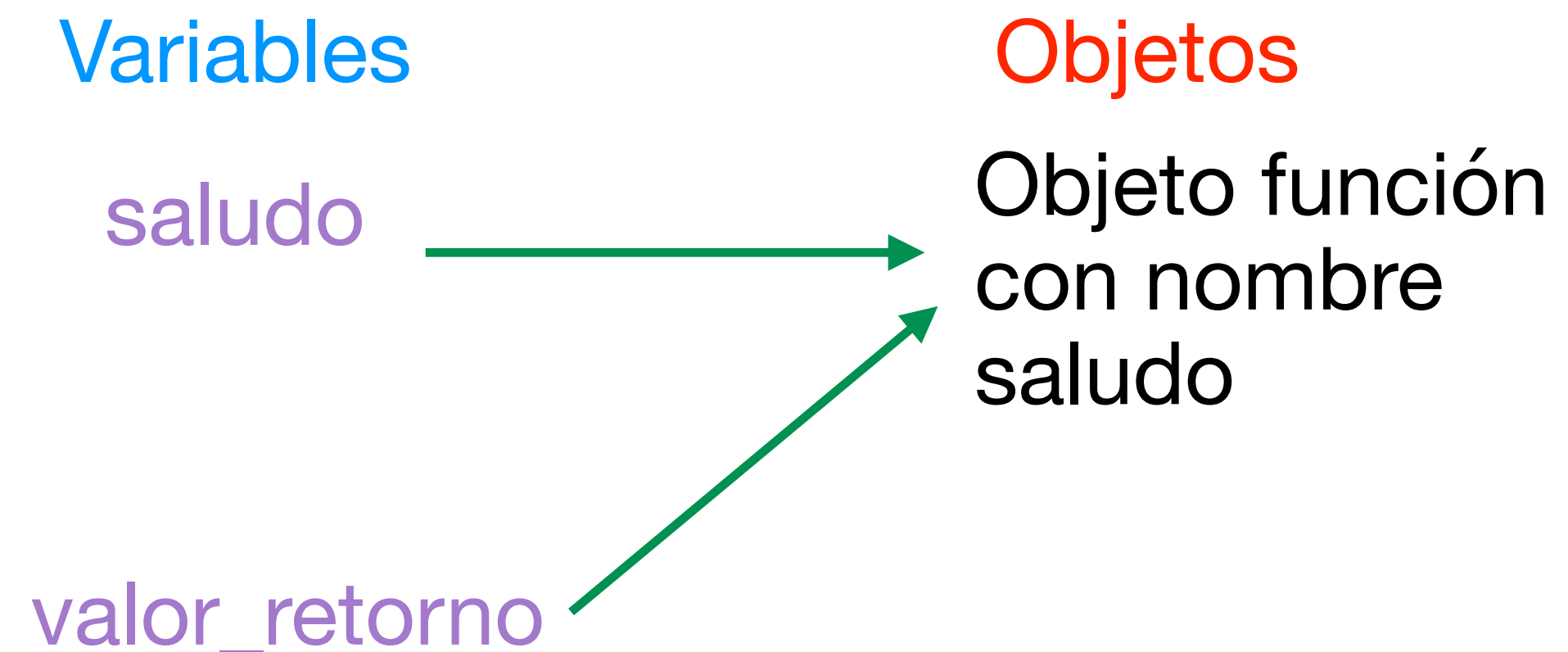
No se hace la llamada a la función

Llamada a la función sin paréntesis

```
def saludo():  
    return('Hola mundo')  
  
la_misma_funcion = saludo  
print(la_misma_funcion())
```

Llamada a la función con
la_misma_funcion()

¿Qué nos da la salida?



Parámetros de la función

Una función puede tener parámetros. Todo lo que va dentro del paréntesis son parámetros de la función.

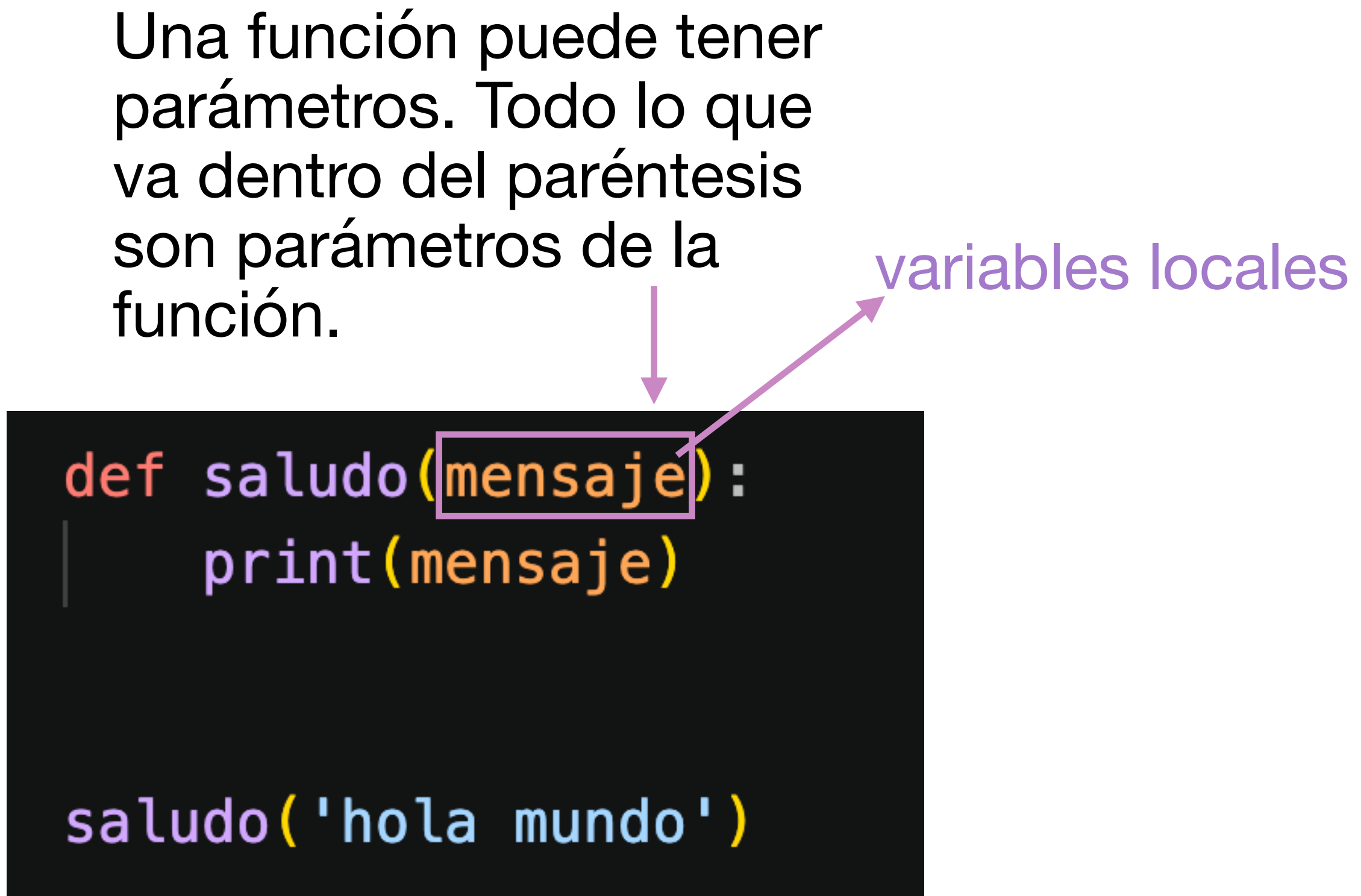


```
def saludo(mensaje):  
    print(mensaje)  
  
saludo('hola mundo')
```

Parámetros de la función

Una función puede tener parámetros. Todo lo que va dentro del paréntesis son parámetros de la función.

variables locales



```
def saludo(mensaje):  
    print(mensaje)
```

```
saludo('hola mundo')
```

Parámetros de la función

Una función puede tener **parámetros**. Todo lo que va dentro del paréntesis son parámetros de la función.

```
def saludo(mensaje):  
    print(mensaje)
```

variables locales

```
saludo('hola mundo')
```

Los **argumentos** son los valores que le pasamos a la función al llamarla. Son **objetos**.

Ejercicio

Escribe una función que reciba dos parámetros y regrese su suma.

1. Llámala para sumar 3 y 5
2. Al resultado anterior, súmale la suma de 4 y 2 (utilizando la función otra vez)

Polimorfismo

1. Escribe una función que reciba dos números flotantes y regrese su suma.
2. Escribe una función que reciba dos números complejos y regrese la suma.
3. Escribe una función que concatene dos cadenas.

```
def suma(x,y):  
    return x + y  
  
suma_resultado = suma(2,3)  
print(suma_resultado)
```

¿Y si quiero una función
que tome los números del
1 al 99 y los sume?

```
num = range(100)

def suma(numeros):
    # sum() espera un iterable
    return sum(numeros)

print(suma(num))
```

¿Y si quiero una función
que tome los números del
1 al 99 y los sume?

```
num = range(100)

def suma(numeros):
    # sum() espera un iterable
    return sum(numeros)

print(suma(num))
print(suma([1,2,3]))
print(suma((1,1,1,1,1)))
```

El valor de retorno puede devolver cualquier tipo de objeto

Ejemplo: tupla

```
def operaciones(x,y):  
    suma = x + y  
    resta = x - y  
    mul = x*y  
  
    return suma, resta, mul  
  
valor_retorno = operaciones(7,9)  
print(type(valor_retorno))  
print(valor_retorno)
```

¿Cuál se la salida del código?

El valor de retorno puede devolver cualquier tipo de objeto

Ejemplo: tupla

```
def operaciones(x,y):  
    suma = x + y  
    resta = x - y  
    mul = x*y  
  
    return suma, resta, mul  
  
valor_retorno = operaciones(7,9)  
print(type(valor_retorno))  
print(valor_retorno)
```

```
def operaciones(x,y):  
    suma = x + y  
    resta = x - y  
    mul = x*y  
  
    return suma, resta, mul  
  
# podemos desempaquetar el valor de retorno  
suma, resta, mul = operaciones(7,9)  
print(suma)
```

¿Cuál se la salida del código?

El valor de retorno puede devolver cualquier tipo de objeto

Ejemplo: tupla

```
def operaciones(x,y):  
    suma = x + y  
    resta = x - y  
    mul = x*y  
  
    return suma, resta, mul  
  
valor_retorno = operaciones(7,9)  
print(type(valor_retorno))  
print(valor_retorno)
```

¿Cuál se la salida del código?

```
def operaciones(x,y):  
    suma = x + y  
    resta = x - y  
    mul = x*y  
  
    return suma, resta, mul  
  
# podemos desempaquetar el valor de retorno  
suma, resta, mul = operaciones(7,9)  
print(suma)
```

Variables con el mismo nombre. No hay problema, unas son globales y otras locales

Scope y namespace (Alcance y espacio de nombres)

```
x = 10  
y = 9
```

```
def f1():  
    x = 'hola'  
    y = 'mundo'  
    z = 2
```

```
print(x)  
print(z)
```

Scope y namespace (Alcance y espacio de nombres)

```
x = 10
```

```
y = 9
```

```
def f1():
```

```
    x = 'hola'
```

```
    y = 'mundo'
```

```
    z = 2
```

```
print(x)
```

```
print(z)
```

Scope y namespace (Alcance y espacio de nombres)

```
x = 10
y = 9

def f1():
    x = 'hola'
    y = 'mundo'
    z = 2

print(x)
print(z)
```

Variables globales

Variables locales. Solo las
puede ver la función

Scope y namespace (Alcance y espacio de nombres)

```
x = 10
```

```
def f1():  
    print(x)
```

```
f1()
```

¿Cuál es la salida?

Scope y namespace (Alcance y espacio de nombres)

```
x = 10
y = 5
z = -1
print('globals:', globals())

def f1():
    a = 'hola'
    b = 'mundo'
    print('locals:', locals())

f1()
```

Nos da las variables
globales

Nos da las variables
locales

Scope y namespace (Alcance y espacio de nombres)

```
x = 100
a = 9
b = 5

def f():
    x = 12
    c = 10
    print(locals())

    def f2():
        d = 9
        print(x)
        print(locals())

    f2()

f()
```

Argumentos y parámetros

```
def suma(a,b,c,d=5,e=9):  
    print(a + b + c + d + e)  
  
# formas de llamar a la función  
suma(1,1,1,1,1) # argumentos posicionales  
suma(2,3,4)      # posicionales y valor por defecto  
  
# argumentos de palabra clave. Las claves deben coincidir  
# con los parámetros de la función  
suma(d=5,b=1,a=-9,c=1,e=2)  
  
# argumentos combinados  
# los argumentos POSICIONALES siempre van primero  
suma(1,2,3,e=3,d=-1)
```

Podemos tener parámetros posicionales o de valor por defecto. Se pueden combinar.

Podemos tener argumentos posicionales, por palabra clave, o combinados.

```
def suma(a,b,c,d=5,e=9):  
    print(a + b + c + d + e)
```

```
suma(1,2,3,e=3,d=-1)
```

En el caso de argumentos posicionales la asignación se hace en orden

```
def suma(a,b,c,d=5,e=9):  
    print(a+b+c+d+e)  
  
suma(1,2,3,e=3,d=-1)
```

En el caso de argumentos posicionales la asignación se hace en orden

Los argumentos de palabra clave se pueden pasar en desorden.

RECORDAR: los argumentos posicionales siempre van primero

Restricciones sobre los argumentos

Nos dice que a y b se tienen que pasar como argumentos posicionales

Nos dice que e si tiene que pasar como argumento de palabra clave

```
def suma(a,b,/,c,d=5,*,e=9):  
    print(a + b + c + d + e)  
  
suma(5,4,5,1,e=-4)
```

Nota: la restricción es sobre los argumentos, no sobre los parámetros

Restricciones sobre los argumentos

Nos dice que a y b se tienen que pasar como argumentos posicionales

Nos dice que e se tiene que pasar como argumento de palabra clave

```
def suma(a=0,b=9,/,c=0,d=5,*,e=9):  
    print(a + b + c + d + e)  
  
suma(5,4,5,1,e=-4)
```

Nota: la restricción es sobre los argumentos, no sobre los parámetros

La restricción es en los argumentos, a y b deben pasarse como posicionales. No hay restricción en los parámetros, a y b son de valor predeterminado.